

SmartBubbleDetect

ABSTRACT

- Air bubbles in intravenous (IV) infusion lines can lead to serious health risks if they enter a patient's bloodstream.
- This project presents the design and development of an IoT-based Ultrasonic Air Bubble Detection System for continuous and non-invasive monitoring of IV tubes.
- The system uses an ultrasonic transmitter and receiver to detect air bubbles by monitoring changes in the received ultrasonic signal.
- The signal is amplified and processed by an STM32 NUCLEO-F207ZG microcontroller, which converts it into a digital value using its ADC and compares it with a configurable threshold stored in EEPROM.
- When an air bubble is detected, the STM32 generates a JSON message containing the Device ID and bubble status, which is transmitted through Ethernet inside STM32 using the MQTT protocol.
- The data is received by a Node-RED dashboard cloud, where the corresponding patient information, device status, and detection results are displayed in real time.
- The system also provides visual (LED) alerts for immediate notification. The proposed solution offers a cost-effective, reliable, and real-time IoT-based monitoring system to improve patient safety during intravenous infusion.

OBJECTIVE

- To develop an IoT-based intravenous (IV) monitoring system for real-time monitoring of IV fluid.
- To detect air bubbles in the IV fluid using an ultrasonic bubble detector.
- To acquire and process sensor data using the STM32 NUCLEO-F207ZG microcontroller.
- To store the latest monitoring data in AT24C256 EEPROM for reliable data retention.
- To transmit the monitored data to the HiveMQ Cloud MQTT broker using the on-board Ethernet interface.
- To visualize the IV monitoring data on the FlowFuse (Node-RED) dashboard for remote monitoring.
- To enable continuous monitoring of the patient's IV condition in real time.
- To improve patient safety by providing timely detection of abnormal conditions, such as air bubbles in the IV line.

FEATURES

Non-Contact Ultrasonic Air Bubble.

- Detects air bubbles without coming into contact with the IV fluid.
- Uses ultrasonic transmitter and receiver technology.

Real-Time Bubble Detection

- Continuously monitors the IV tube.
- Detects bubbles within milliseconds.

STM32-Based Embedded Processing Stm32 Nucleo-F207ZG

- The STM32 NUCLEO-F207ZG features an ARM Cortex-M3 microcontroller running at up to 120 MHz with 1 MB Flash and 128 KB SRAM. It has built-in 10/100 Mbps Ethernet with an on-board Ethernet PHY and RJ45 connector, eliminating the need for an external Ethernet module. The board also includes USB OTG, ST-LINK/V2-1 debugger, Arduino-compatible headers, and multiple UART, SPI, I²C, CAN, ADC, and DAC peripherals.

Configurable Threshold

- Bubble detection threshold can be adjusted.
- Threshold value can be stored in EEPROM

EEPROM-Based Configuration (AT24C256)

- Stores Device ID.
- Stores threshold value.
- Retains the values even after power is turned OFF.

UART Interface

- Used for debugging, calibration, and monitoring ADC values using PuTTY.

MQTT Communication

- Publishes bubble detection status to the MQTT broker using lightweight IoT communication.

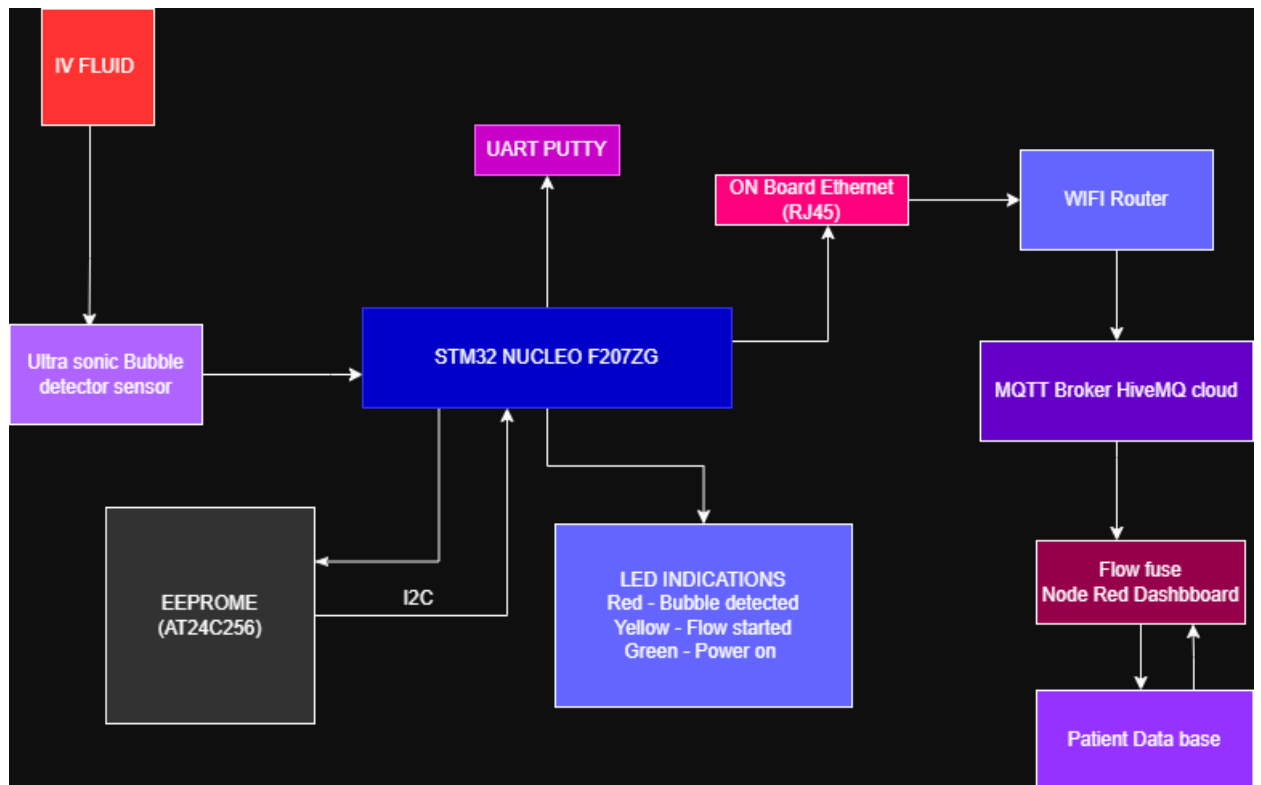
LED Indication

- Provides a visual indication when a bubble is detected.
- Red - Bubble detected
- Green - normal
- Yellow - status

Technologies Used

- STM32 Microcontroller inbuilt ethernet
- Ultrasonic Transmitter & Receiver
- EEPROM
- UART
- MQTTX
- JSON
- Node-RED Dashboard

BLOCK DIAGRAM



WORKING

- **Ultrasonic sensor**

- It consists of a transmitter and receiver; the transmitter emits ultrasonic waves through the IV tube, and when an air bubble is present, the received signal becomes weaker due to reflection and scattering, allowing the STM32 to detect the bubble
- STM32 Data Processing-The STM32 receives the distance value and compares it with predefined thresholds.
- The STM32 writes the latest sensor reading (distance, IV level, patient ID, etc.) to the AT24C256 EEPROM through the I²C interface.
- After writing, the STM32 reads back the stored data from the EEPROM to verify that it has been written correctly.

- **UART Debugging (PuTTY)**

The STM32 transmits the sensor readings, such as the patient ID, IV fluid level, and distance, to a PC through the UART interface, where they are displayed in PuTTY for debugging and monitoring purposes.

- **JSON Payload Creation**

The STM32 converts the processed sensor data into a JSON format containing details such as the patient ID, bed number, IV fluid level, distance, LED status, and timestamp, and stores it as a character string before transmission.

- **MQTTX Publishing**

The STM32 publishes the generated JSON payload to a predefined MQTT topic, such as hospital/icu/bed12/ivlevel, using the MQTT protocol.

- **Ethernet Communication**

Using the built-in Ethernet interface of the **STM32 NUCLEO-F207ZG**, the JSON data is transmitted through the **RJ45 Ethernet port** to the hospital router and then forwarded over the Internet to the HiveMQ Cloud MQTT Broker

- **HiveMQ Cloud Broker:**

The HiveMQ Cloud MQTT Broker receives the published JSON message from the STM32 device, temporarily processes and stores it in the message queue, and forwards it to all clients subscribed to the corresponding MQTT topic. This ensures secure, reliable, and real-time communication between publishers and subscribers.

- **FlowFuse Node-RED**

The Node-RED application hosted on FlowFuse subscribes to the MQTT topic and automatically receives the JSON data whenever a new message is published.

- **JSON Parsing**

Node-RED extracts individual fields such as the patient ID, distance, IV fluid level, and LED status from the received JSON payload for further processing.

- **Dashboard Update**

The extracted data is displayed on the Node-RED dashboard in real time

Work completion update:

Cloud Communication Testing using MQTTX, HiveMQ Cloud and FlowFuse

Step 1: HiveMQ Cloud Setup

- Created a HiveMQ Cloud account.
- A new HiveMQ Cloud broker was created in the HiveMQ Cloud platform.
- Obtained the **server address (Host ID)**.
- Created a **username and password** for secure access.

Step 2: MQTTX Configuration (Publisher)

- Installed MQTTX.
- Created a new MQTT connection.
- Entered:
 - HiveMQ Cloud server address (Host)
 - Port **8883**
 - Username
 - Password
- Connected to the HiveMQ Cloud broker.
- Published JSON data to the topic (e.g., project/test).

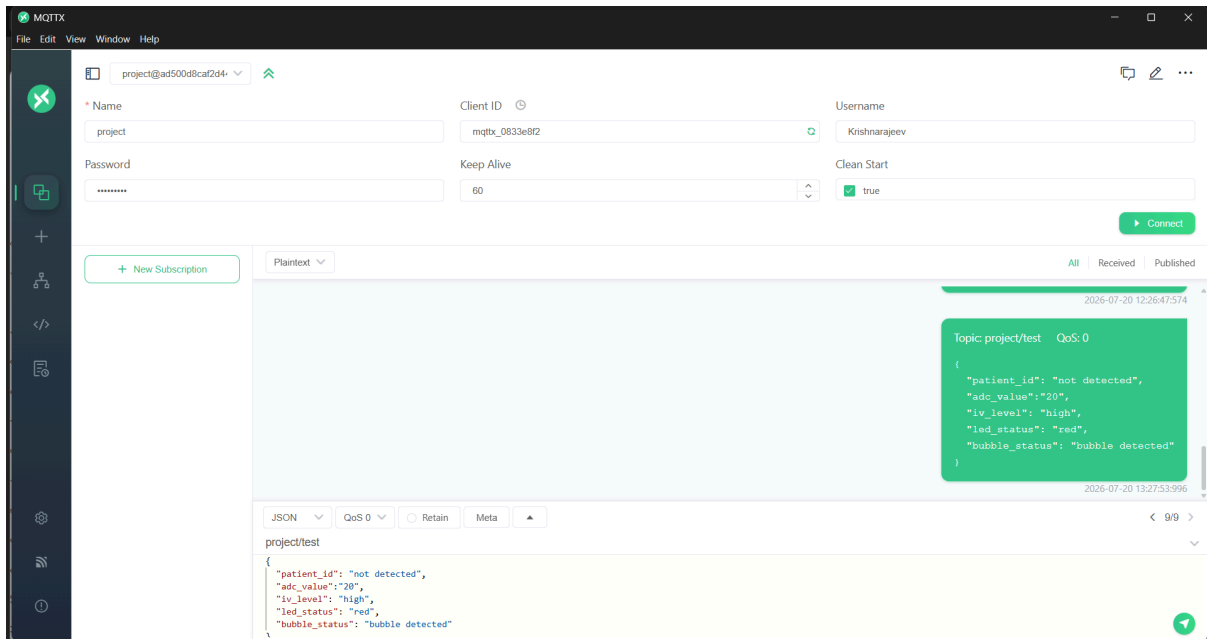
Step 3: FlowFuse Configuration (Subscriber)

- Created a new FlowFuse project.
- Added an **MQTT Input (MQTT In)** node to the workspace.
- Configured the MQTT Input node with:
 - **HiveMQ Cloud server address**
 - **Port: 8883**

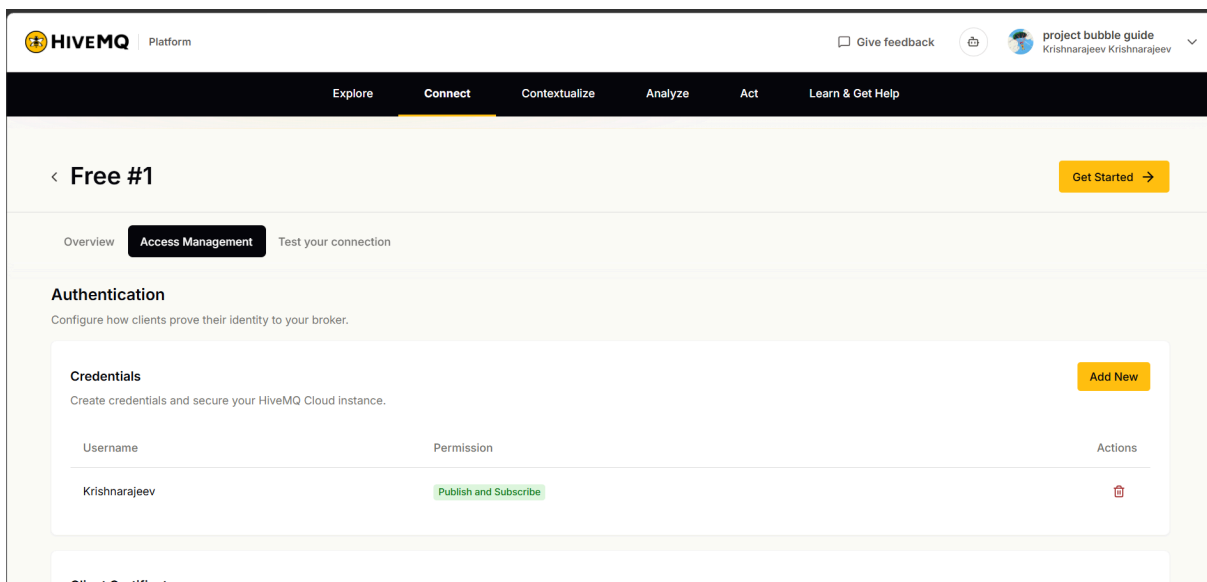
- **Username**
- **Password**
- **MQTT Topic:** project/test
- Connected the **MQTT Input** node to a **Debug** node.
- Deployed the flow.
- Verified that the published messages from MQTTeX were successfully received in the Debug window.

Step 4: Communication Verification

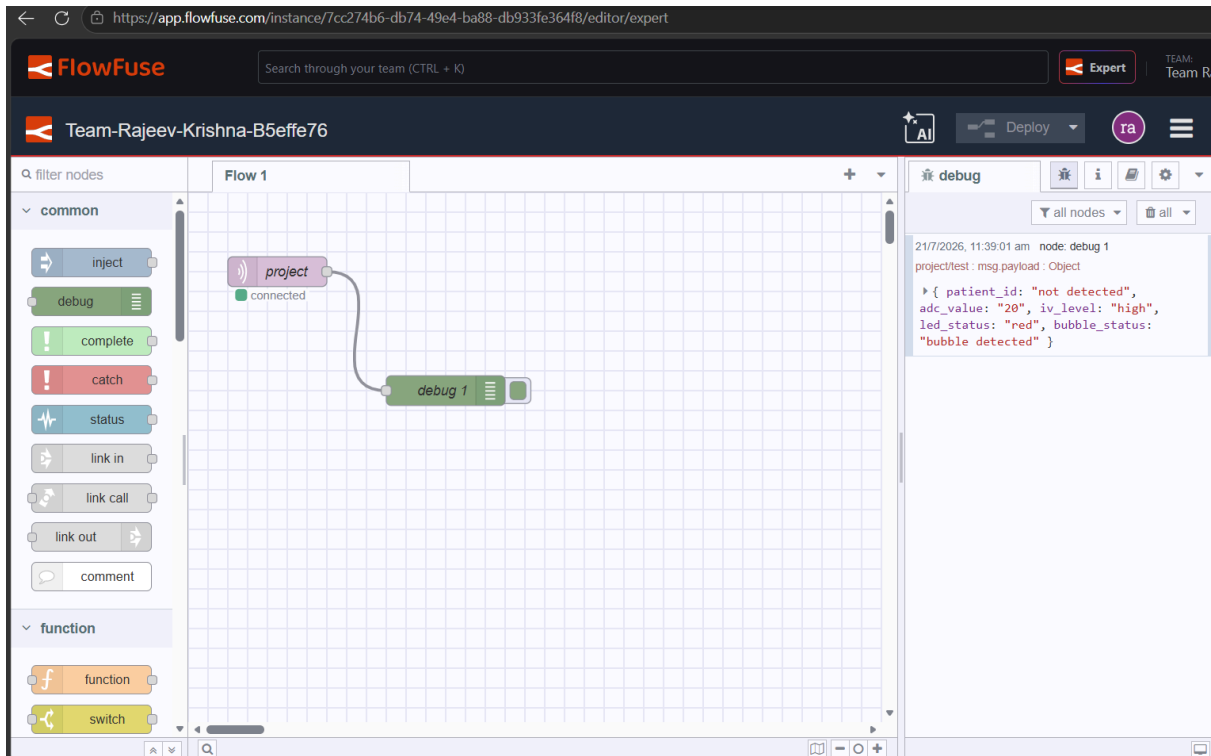
- Published test messages from MQTTeX.
- HiveMQ Cloud received the messages.
- FlowFuse subscribed to the topic and displayed the received JSON data.
- This verified that the MQTT communication was working correctly.



MQTTX configured as publisher



HIVEMQ configured as broker



Flow fuse configured as Subscriber

Conclusion

This project aims to develop an IoT-based IV monitoring system for real-time detection of air bubbles in IV fluid and remote monitoring of IV status. The proposed system will utilize the STM32 NUCLEO-F207ZG microcontroller to acquire and process sensor data, store the latest readings in EEPROM, and transmit the information to the HiveMQ Cloud MQTT broker through the on-board Ethernet interface. The received data will be displayed on the FlowFuse (Node-RED) dashboard, enabling healthcare professionals to monitor the IV condition remotely. Upon completion, the proposed system is expected to enhance patient safety, reduce the need for continuous manual supervision, and provide a reliable and cost-effective solution for smart healthcare applications.

